

Project BOB

Developer Step-by-Step Guide

Bonga na BOB -- Kenyan Betting Intelligence Platform How every file was built, why each decision was made, and what every line of code is actually doing.

Author	Data-Amigo
Version	0.1 -- May 2026
Audience	Developer building the project step by step
Stack	Python 3.11 Playwright BeautifulSoup ReportLab

Table of Contents

How to Use This Guide	3
Step 0 -- Project Setup & Wiring	3
0.1 pyproject.toml -- What it declares and why	3
0.2 Folder layout -- src-layout explained	4
0.3 Virtual environment & installing dependencies	4
Step 1 -- The Browser Engine (browser.py)	5
1.1 Why Playwright and not requests/httpx	5
1.2 BrowserFetchResult -- the contract object	5
1.3 fetch_page() -- walkthrough line by line	6
1.4 Security challenge detection	7
1.5 Helper functions	8
1.6 How to run it yourself	8
Step 2 -- The Source Registry (sources.py)	9
2.1 Why a registry at all	9
2.2 SourceTarget dataclass	9
2.3 SourceConfig dataclass	10
2.4 The SOURCES dictionary	10
2.5 Lookup helpers	11
Step 3 -- Data Models (models/)	11
3.1 Why models are defined before parsers	11
3.2 ForebetFootballPrediction	12
3.3 ForebetBasketballPrediction	12
3.4 SportPesaFootballOdds	13
3.5 PolymarketMarket	13
3.6 PolymarketFetchConfig & PolymarketRawResponse	14
Step 4 -- The Forebet Parser (parsers/forebet.py)	14
4.1 How we learned the page structure	14
4.2 Tokenisation strategy	15

4.3 Index constants	15
4.4 parse_forebet_football()	16
4.5 parse_forebet_basketball()	16
4.6 Confidence scoring	17
Step 5 -- The SportPesa Parser (parsers/sportpesa.py)	17
5.1 The two-layer problem	17
5.2 Extracting fixture identity from JSON-LD	18
5.3 Extracting odds from the DOM with regex	18
5.4 Joining the two layers	19
Step 6 -- The Polymarket API Client	19
6.1 Why API and not browser for Polymarket	19
6.2 The Gamma API endpoints	20
6.3 The normalizer -- parsers/polymarket.py	20
Step 7 -- Inspection Scripts & Smoke Tests	21
Step 8 -- What Comes Next	22

How to Use This Guide

This is not a reference manual -- it is a build diary. It explains every file in the project in the exact order things were built, and at every step it answers three questions:

- * WHAT was built and what does it do?
- * WHY was it built this way and not another way?
- * HOW does the code actually work, line by line?

TIP

Read this guide alongside the actual source files open in your editor. The guide explains the reasoning; the source file shows the full code. Together they give you complete understanding.

Each section uses the same four visual cues consistently:

WHY

The reasoning behind a design decision -- why this approach was chosen over the alternatives.

NOTE

An important detail to understand before moving on.

WARNING

Something that can go wrong and how to avoid it.

TIP

A practical shortcut or useful habit when working with this module.

0

Project Setup & Wiring

pyproject.toml | folder layout | virtual environment | dependencies

0.1 pyproject.toml -- What it declares and why

pyproject.toml is the single source of truth for the entire Python project. It replaces the older setup.py and requirements.txt pattern. Everything the project needs to be installed, built, and linted is declared here.

```
pyproject.toml

[build-system]
requires = ["setuptools>=69", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "project-bob"
version = "0.1.0"
description = "Bonga na BOB -- scraper-first betting intelligence platform."
requires-python = ">=3.11"
dependencies = [
    "beautifulsoup4>=4.12.3", # HTML parsing for Forebet/SportPesa
    "httpx>=0.27.0", # async-capable HTTP (future use)
    "openai>=1.30.0", # LLM calls for the Decision Agent
    "playwright>=1.44.0", # browser engine for JS-rendered pages
    "pydantic-settings>=2.2.1", # .env + settings management
    "python-dotenv>=1.0.1", # loads .env for local dev
    "python-telegram-bot>=21.1.1", # Telegram delivery layer
    "sqlalchemy>=2.0.30", # ORM for PostgreSQL (planned)
    "psycopg[binary]>=3.1.19", # PostgreSQL driver Python 3.11
    "streamlit>=1.33.0" # scraper UI and dashboard
]

[project.optional-dependencies]
dev = ["pytest>=8.2.0", "ruff>=0.4.4"]

[tool.setuptools]
package-dir = {"" = "src"} # package lives in src/, not root

[tool.ruff]
line-length = 100
target-version = "py311" # linter targets 3.11 syntax
```

WHY

We use src-layout (package-dir = "" = 'src') so that the installed package and the local source folder are never confused. If the package root were the project root, Python would import from the local folder even after installing -- hiding any packaging bugs until deployment.

0.2 Folder Layout -- src-layout explained

The project is split into three zones. Understanding which zone a file lives in tells you immediately what it does and how to run it.

src/project_bob/	The installable Python package. All production code lives here. Never run files in here directly -- import them.
scripts/	Standalone developer scripts. These are not part of the package. Run them directly with: <code>python scripts/smoke_fetch.py</code>
docs/	Markdown architecture and roadmap documents. For humans, not for Python.

0.3 Virtual Environment & Installing Dependencies

Every command below is run once from the project root when setting up a new development machine.

```
# 1. Create the virtual environment (Python 3.11 required)
python -m venv .venv

# 2. Activate it
# Windows:
.venv\Scripts\activate
# macOS / Linux:
source .venv/bin/activate

# 3. Install the project and all its dependencies (editable mode)
# -e means changes to src/ are reflected immediately without re-installing
pip install -e ".[dev]"

# 4. Install the Playwright browser binaries (Chromium only)
playwright install chromium

# 5. Verify the install
python -c "from project_bob.scrapers.browser import fetch_page; print('OK!')"
```

NOTE

`pip install -e '[dev]'` installs both production and development dependencies (pytest, ruff). The `-e` flag means the package is installed in 'editable' mode -- Python points directly at your `src/` folder so you see code changes immediately without reinstalling.

WARNING

`playwright install chromium` downloads ~170MB of browser binaries. Run this once and it is cached. On Windows, use `setup_playwright.ps1` in the `scripts/` folder if the command above does not work.

1

The Browser Engine

`src/project_bob/scrapers/browser.py` -- built first, used by everything

This was the very first file written in the project. Every other scraper depends on it. Its job is simple: open a URL in a real browser, wait for it to render, and hand back the HTML -- or an error -- in a predictable shape.

1.1 Why Playwright and not requests or httpx?

The two main target sites -- Forebet and SportPesa -- are single-page applications built with heavy JavaScript. If you fetch their URLs with requests or httpx you receive the server-rendered HTML shell, which contains almost no match data. The actual content is injected by JavaScript after the page loads in a browser.

Tool	What you get	Works for BOB?
requests / httpx	Raw server response -- HTML shell only, no JS execution	No -- JS content missing
Selenium	Real browser, but slow API and complex setup	Possible but painful
Playwright (chosen)	Real Chromium browser, modern async/sync API, fast, good snapshot/screenshot tools	Yes

1.2 BrowserFetchResult -- the contract object

Before writing a single line of scraping logic, we designed the return type. This is the most important design decision in the file.

WHY

Every function that calls `fetch_page()` receives the same `BrowserFetchResult` object whether the page loaded successfully or not. This means callers never need `try/except` blocks -- they just check `result.ok`. The Streamlit UI, the pipeline, and tests all speak the same language.

```
scrapers/browser.py -- BrowserFetchResult

@dataclass(slots=True) # slots=True saves memory -- no __dict__ per instance
class BrowserFetchResult:
    url: str # what we tried to open
    status: str # "success" or "failed"
    started_at: datetime # UTC -- when the browser work began
    finished_at: datetime # UTC -- when it ended
    duration_ms: int # total wall-clock time in milliseconds
    title: str = "" # the browser tab title after render
    html: str = "" # the full rendered HTML
    html_length: int = 0 # len(html) -- quick sanity check
    snapshot_path: str | None = None # path if we saved the HTML to disk
```

```
screenshot_path: str | None = None # path if we saved a screenshot
warnings: list[str] = field(default_factory=list) # non-fatal issues
error: str | None = None # set when status == "failed"

@property
def ok(self) -> bool:
    return self.status == "success" # lets callers write: if result.ok
```

1.3 fetch_page() -- walkthrough section by section

This is the only public function in browser.py. Everything else is a private helper called only from inside this function.

Signature & defaults

```
def fetch_page(
    url: str,
    *, # everything after * must be passed by keyword
    timeout_ms: int = 60_000, # 60 seconds -- enough for slow JS sites
    wait_until: str = "domcontentloaded", # don't wait for network idle
    settle_ms: int = 3_000, # wait 3 extra seconds after DOM loads
    headless: bool = True, # False = show a visible browser window
    snapshot_path: str | Path | None = None, # save HTML here if set
    screenshot_path: str | Path | None = None, # save screenshot here if set
) -> BrowserFetchResult:
```

WHY

wait_until='domcontentloaded' is chosen over 'networkidle' deliberately. Betting sites run continuous background requests (ads, trackers, live scores). 'networkidle' would wait for all of these to stop -- which may never happen. domcontentloaded fires as soon as the DOM is ready, then settle_ms gives JS time to inject the real content we need.

Timing setup

```
started_at = datetime.now(UTC) # human-readable timestamp for logs
start = perf_counter() # high-precision timer for duration_ms
warnings: list[str] = [] # collect non-fatal issues without crashing
```

We capture two time references: a UTC datetime for human-readable logs and perf_counter() for precise millisecond measurement. They serve different purposes -- the datetime goes into the database; the perf_counter drives duration_ms.

Browser lifecycle

```
with sync_playwright() as playwright:
    # sync_playwright() starts the Playwright controller process
    browser = playwright.chromium.launch(headless=headless)
    # chromium.launch() starts a real Chromium browser instance
    page = browser.new_page()
```



```

# new_page() opens a blank tab inside that browser
page.goto(url, wait_until=wait_until, timeout=timeout_ms)
# goto() navigates to the URL and waits for the chosen load event
if settle_ms > 0:
    page.wait_for_timeout(settle_ms)
# wait_for_timeout() is a simple sleep inside the browser event loop
# giving JS frameworks time to inject content after DOM is ready
title = page.title() # grab the tab title
html = page.content() # grab the full rendered HTML
saved_screenshot_path = _save_screenshot(screenshot_path, page)
browser.close() # always close to free memory

```

NOTE

The 'with sync_playwright() as playwright' context manager guarantees cleanup. Even if `page.goto()` raises an exception, Playwright's internal processes are shut down properly. This is why we catch exceptions outside the 'with' block.

Result assembly

```

saved_snapshot_path = _save_snapshot(snapshot_path, html)
finished_at = datetime.now(UTC)
duration_ms = int((perf_counter() - start) * 1000)
if not html.strip():
    warnings.append("Page returned empty HTML.") # suspicious but not fatal
warnings.extend(_detect_security_challenge(title=title, html=html))
return BrowserFetchResult(
    url=url, status="success",
    started_at=started_at, finished_at=finished_at,
    duration_ms=duration_ms, title=title, html=html,
    html_length=len(html), snapshot_path=saved_snapshot_path,
    screenshot_path=saved_screenshot_path, warnings=warnings,
)

```

Error handling

```

except PlaywrightTimeoutError as exc:
    return _failed_result(url, started_at, start, f"Timed out loading page: {exc}")
except Exception as exc:
    return _failed_result(url, started_at, start, str(exc))

```

WHY

We catch exceptions and return a failed result object rather than re-raising. This is a deliberate choice: the Streamlit UI and any future pipeline code can always check `result.ok` without wrapping every call in `try/except`. Exceptions stay inside this module -- they never leak outward.

1.4 Security Challenge Detection

Forebet and SportPesa are protected by Cloudflare. When running headless, Playwright sometimes receives an anti-bot challenge page instead of real content. The HTML is valid and the status is 'success' -- but the data is useless.

```
# Title phrases are the strongest signal
SECURITY_CHALLENGE_TITLE_PHRASES = (
    "just a moment", # Cloudflare's most common title
    "checking your browser",
    "security verification",
)

# HTML phrases are a fallback -- only phrases that strongly indicate a challenge
SECURITY_CHALLENGE_HTML_PHRASES = (
    "performing security verification",
    "verify you are not a bot",
    "security service to protect against malicious bots",
    "cf-challenge", # Cloudflare challenge container ID
    "cf-turnstile", # Cloudflare Turnstile widget
    "challenge-platform", # Cloudflare challenge platform marker
)
```

NOT E

This detection does NOT bypass or circumvent any security system. It is purely informational -- it adds a warning to the result so the developer or UI knows that the HTML collected may not contain real betting data. The actual solution is to run in headed mode (headless=False) which Forebet and SportPesa require.

WHY

We avoid broad phrases like 'cloudflare' in HTML detection because real, successfully-loaded pages often still contain Cloudflare scripts for analytics after the challenge has already passed. Broad detection would produce false positives.

1.5 Helper Functions

`_save_snapshot(snapshot_path, html)`

If `snapshot_path` is provided, writes the rendered HTML to disk. Used for debugging -- you can open the file in a browser to see exactly what Playwright collected.

```
_save_snapshot(snapshot_path, html):
    if snapshot_path is None:
        return None
    path = Path(snapshot_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(html, encoding='utf-8')
    return str(path)
```

`_save_screenshot(screenshot_path, page)`

Captures a full-page PNG screenshot of whatever the browser rendered. Invaluable when you need to see a Cloudflare challenge page visually.

```
_save_screenshot(screenshot_path, page):
    if screenshot_path is None:
        return None
    path = Path(screenshot_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    page.screenshot(path=str(path), full_page=True)
    return str(path)
```

`_failed_result(url, started_at, start, error)`

Builds a consistent BrowserFetchResult with status='failed'. Keeps failure responses in exactly the same shape as success responses.

```
_failed_result(url, started_at, start, error):
    finished_at = datetime.now(UTC)
    duration_ms = int((perf_counter() - start) * 1000)
    return BrowserFetchResult(
        url=url, status='failed',
        started_at=started_at, finished_at=finished_at,
        duration_ms=duration_ms, error=error,
    )
```

1.6 How to Run It Yourself

```
# Open a Python REPL or run as a script after activating your venv
from project_bob.scrapers.browser import fetch_page

result = fetch_page(
    "https://www.forebet.com/en/football-tips-and-predictions-for-today",
    headless=False, # headed mode -- avoids Cloudflare challenge
    settle_ms=15_000, # Forebet needs 15s for JS to fully render
    snapshot_path="debug/forebet_snapshot.html",
    screenshot_path="debug/forebet_screenshot.png",
)

print(result.ok) # True or False
print(result.duration_ms) # how long it took in ms
print(result.warnings) # any security challenge warnings
print(result.html_length) # how much HTML we got
# If ok, result.html contains the full rendered page
```

TIP

Always start with `headless=False` when testing a new source for the first time. You can watch the browser open, the page render, and any Cloudflare challenges appear. Once you know the site works, you can try `headless=True`.

2

The Source Registry

src/project_bob/scrapers/sources.py -- single place to configure every data source

2.1 Why a Registry at All?

Without a registry, every scraper would hardcode its own URL, timeout, and headless setting. When Forebet changes its URL or you add a new source, you would need to hunt through every scraper file to update it.

WHY

The registry pattern gives us one place to add, remove, or reconfigure any source. The Streamlit UI, smoke tests, and pipeline all call `get_source_config()` -- they never hardcode URLs or settings themselves. This is the single source of truth for 'which sites does this project know about?'

2.2 SourceTarget -- One Page Under a Source

A source like Forebet has multiple useful pages -- football today, basketball today, maybe tomorrow's predictions. Each page is a `SourceTarget`.

```
scrapers/sources.py -- SourceTarget

@dataclass(frozen=True, slots=True) # frozen = immutable after creation
class SourceTarget:
    name: str # internal key: "football_today"
    display_name: str # shown in UI: "Football Predictions Today"
    url: str # exact URL to fetch
    sport: str # "football", "basketball", "market_intelligence"
    description: str # one-sentence explanation

    # Example:
    SourceTarget(
        name="football_today",
        display_name="Football Predictions Today",
        url="https://www.forebet.com/en/football-tips-and-predictions-for-today",
        sport="football",
        description="Forebet football predictions for today's matches.",
    )
```

NOTE

`frozen=True` means no one can accidentally change a source target at runtime. These are configuration constants, not mutable state. If you need to add a new target, edit the `SOURCES` dictionary -- do not mutate an existing object.

2.3 SourceConfig -- One Supported Website

```
@dataclass(frozen=True, slots=True)
```

```

class SourceConfig:
    name: str # internal key: "forebet"
    display_name: str # shown in UI: "Forebet"
    default_target: str # which SourceTarget to use by default
    targets: dict[str, SourceTarget] # all available targets for this source
    description: str # one-sentence role description
    default_wait_until: str = "domcontentloaded"
    default_settle_ms: int = 3_000 # extra JS render wait
    default_headless: bool = True # False for Forebet/SportPesa

    @property
    def default_url(self) -> str:
        # convenience: returns the URL for the default target
        return self.targets[self.default_target].url

```

2.4 The SOURCES Dictionary

SOURCES is the actual registry -- a dict mapping source name strings to SourceConfig objects. Here is what each entry looks like and why the settings differ between sources:

```

scrapers/sources.py -- SOURCES dict

SOURCES: dict[str, SourceConfig] = {
    "forebet": SourceConfig(
        name="forebet",
        default_target="basketball_today",
        default_settle_ms=15_000, # needs 15s -- heavy JS framework
        default_headless=False, # Cloudflare blocks headless Chromium
        targets={
            "football_today": SourceTarget(...),
            "basketball_today": SourceTarget(...),
        },
    ),
    "sportpesa": SourceConfig(
        name="sportpesa",
        default_settle_ms=10_000, # 10s for React app to mount
        default_headless=False, # also has Cloudflare protection
        targets={"football_today": SourceTarget(...)},
    ),
    "polymarket": SourceConfig(
        name="polymarket",
        default_settle_ms=5_000,
        default_headless=True, # API source -- browser only for the UI
        targets={"markets": SourceTarget(...)},
    ),
}

```

}

NOTE

Forebet and SportPesa use headless=False because Cloudflare's bot detection triggers in headless mode during development. Polymarket is headless=True because we use it primarily via API -- the browser target exists for future use if the API changes.

2.5 Lookup Helpers

```
def get_source_config(source_name: str) -> SourceConfig:
    try:
        return SOURCES[source_name]
    except KeyError as exc:
        supported = ", ".join(sorted(SOURCES))
        raise ValueError(
            f"Unsupported source '{source_name}'. Supported: {supported}"
        ) from exc

def get_source_target(source: SourceConfig, target_name: str | None = None) -> SourceTarget:
    selected_target = target_name or source.default_target
    try:
        return source.targets[selected_target]
    except KeyError as exc:
        supported = ", ".join(sorted(source.targets))
        raise ValueError(
            f"Unsupported target '{selected_target}' for '{source.name}'. "
            f"Supported: {supported}"
        ) from exc
```

WHY

These helpers raise ValueError with a useful message listing the valid options. This is better than a raw KeyError because the error message tells the developer exactly what went wrong and what to use instead -- no documentation needed.

3

Data Models

src/project_bob/models/ -- define the shape of data BEFORE writing parsers

3.1 Why Models Are Defined Before Parsers

It might seem natural to write the parser first, then figure out what fields you need. We did the opposite deliberately: models first, parsers second.

WHY

Defining the model first forces you to answer 'what information do I actually need from this site?' before writing a single line of parsing code. This avoids writing a parser that extracts 40 fields when you only need 12, and it gives you a clear target to write toward. The model is the contract -- the parser is just the mechanism that fills it.

NOT E

All models use `@dataclass(slots=True)`. The `slots=True` argument tells Python not to create a `__dict__` for each instance -- it uses fixed memory slots instead. This matters when you have thousands of match records in memory at once.

3.2 ForebetFootballPrediction

This model represents one row from Forebet's football predictions page. Notice that it has a `remaining_tokens` field -- this is intentional.

```
models/forebet.py -- ForebetFootballPrediction

@dataclass(slots=True)
class ForebetFootballPrediction:
    source: str # always "forebet" -- useful when mixing sources
    sport: str # always "football"
    league: str # e.g. "It1" for Serie A
    home_team: str
    away_team: str
    event_datetime: str # raw string -- we don't parse datetime yet
    prob_1: int | None # home win probability as percentage
    prob_x: int | None # draw probability
    prob_2: int | None # away win probability
    pred_outcome: str | None # "1", "X", or "2"
    predicted_home_score: int | None
    predicted_away_score: int | None
    correct_score_text: str | None # e.g. "1 - 2"
    avg_goals: float | None
    weather: str | None
    coef_1: float | None # coefficient for home win
    coef_x: float | None # coefficient for draw
    coef_2: float | None # coefficient for away win
```



```
coef_extra: float | None
remaining_tokens: list[str] # everything after the stable core
raw_text: str # original row text -- always kept for debugging
confidence: float # 1.0 = structure confirmed, 0.75 = uncertain
```

WHY

remaining_tokens stores everything in the row that we have not yet mapped to a named field. Forebet rows contain live match scores and quarter data that we did not fully understand at the time of writing. Rather than discard this data, we keep it for future parsing without breaking the existing stable structure.

3.3 SportPesaFootballOdds

This model captures ten odds values across four betting markets from a single SportPesa fixture. The fields map exactly to what SportPesa displays on screen.

```
models/sportpesa.py -- SportPesaFootballOdds

@dataclass(slots=True)
class SportPesaFootballOdds:
    source: str; sport: str; league: str
    event_datetime: str # "DD/MM/YY - HH:MM" format as shown on page
    game_id: str # SportPesa's internal numeric ID
    home_team: str; away_team: str
    # 3-Way market
    home_odds: float | None; draw_odds: float | None; away_odds: float | None
    # Double Chance market
    home_or_draw_odds: float | None
    draw_or_away_odds: float | None
    home_or_away_odds: float | None
    # Over/Under 2.5
    over_2_5_odds: float | None; under_2_5_odds: float | None
    # Both Teams To Score
    btts_yes_odds: float | None; btts_no_odds: float | None
    game_url: str | None # full URL to the fixture's markets page
    raw_text: str
    confidence: float # 0.9 for V1 -- positional join is not perfect
```

3.4 PolymarketMarket

This is the richest model because the Polymarket Gamma API returns detailed structured data. Notice the `outcome_map()` helper method -- models can have methods, they are not just passive data containers.

```
models/polymarket.py -- PolymarketMarket

@dataclass(slots=True)
class PolymarketMarket:
    source: str # "polymarket"
    market_id: str # unique Polymarket identifier
    event_id: str | None # parent event -- used for category enrichment
    question: str # e.g. "Will Arsenal win the Premier League?"
    slug: str | None # URL slug: "arsenal-win-premier-league"
    category: str | None # "Sports", "Politics", "Crypto", etc.
    subcategory: str | None
    tags: list[str] # flexible labels for filtering
    description: str | None
    start_date: str | None; end_date: str | None
    active: bool; closed: bool; archived: bool
    outcomes: list[str] # e.g. ["Yes", "No"]
    outcome_prices: list[float] # e.g. [0.68, 0.32] -- parallel to outcomes
    liquidity: float | None
    volume: float | None
    open_interest: float | None
    market_type: str | None # AMM or order-book style
    raw_record: dict # full original JSON -- never discard
    confidence: float = 1.0

    def outcome_map(self) -> dict[str, float | None]:
        # Zips outcomes and prices into {"Yes": 0.68, "No": 0.32}
        return {
            outcome: (self.outcome_prices[i] if i < len(self.outcome_prices) else None)
            for i, outcome in enumerate(self.outcomes)
        }
```

WHY

The `raw_record` field stores the entire original JSON payload from the Gamma API. This is intentional. As we understand more about Polymarket's data, we can extract additional fields from `raw_record` without needing to re-fetch the data. It also means debugging is always possible -- you can inspect what the API sent.

3.5 PolymarketFetchConfig & PolymarketRawResponse

```
models/polymarket_fetch.py

@dataclass(slots=True)
class PolymarketFetchConfig:
```

```
result_limit: int = 20 # how many records to return after filtering
scan_limit: int = 100 # how many raw records to fetch from API
active_only: bool = True # filter to active markets only
category: str | None = None # e.g. "sports" -- optional filter

# scan_limit > result_limit because category filtering happens AFTER fetching.
# If you want 20 sports markets, you might need to scan 100 mixed markets first.

@dataclass(slots=True)
class PolymarketRawResponse:
    markets: list[dict] # raw market payloads from /markets endpoint
    events: list[dict] # raw event payloads from /events endpoint
    # We keep markets and events separate -- the parser joins them
```

4

The Forebet Parser

src/project_bob/parsers/forebet.py -- HTML to structured predictions

4.1 How We Learned the Page Structure

Before writing the parser, we ran the browser engine to save a raw HTML snapshot of the Forebet page. We then used the inspection scripts to understand the DOM.

```
scripts/inspect_forebet_snapshot.py

# scripts/inspect_forebet_snapshot.py -- run this first to see the raw structure
from project_bob.scrapers.browser import fetch_page

result = fetch_page(
    "https://www.forebet.com/en/football-tips-and-predictions-for-today",
    headless=False, settle_ms=15_000,
    snapshot_path="debug/forebet_football.html"
)
print(f"Got {result.html_length} chars in {result.duration_ms}ms")

# Then open debug/forebet_football.html in your browser and inspect the DOM.
# Key finding: each match row is a <div class="rcnt"> element.
# The row's text content -- when extracted with get_text(' | ', strip=True)
# -- produces a pipe-separated string we can split into tokens.
```

TIP

The inspection scripts in scripts/ are how every parser was developed. Run them first, examine the snapshot, find the CSS selectors, then write the parser. Never try to write a parser without first looking at the raw HTML.

4.2 Tokenisation Strategy

Once we identified the row structure, we chose the simplest possible parsing approach: flatten each row to text and split on the pipe separator.

```
# In the parser, each row is processed like this:
row_element = soup.select("div.rcnt")[0] # one match row

# BeautifulSoup's get_text with separator flattens the row:
row_text = row_element.get_text(" | ", strip=True)

# Result: "It1 | Arsenal | Chelsea | 15/05/26 14:00 | 58 | 22 | 20 | 1 | 2 | - | 0 | ..."
# ^league ^home ^away ^datetime ^p1 ^px ^p2 ^pred ^score...

# Then split on "|" to get individual tokens:
tokens = [t.strip() for t in row_text.split("|") if t.strip()]
# tokens[0] = "It1", tokens[1] = "Arsenal", tokens[4] = "58", etc.
```

WHY

Pipe-separated tokenisation was chosen over CSS selector per-field extraction because Forebet's row structure is consistent in order but not in class names. The pipe separator from `get_text()` is reliable across different row states (upcoming, live, finished). Index-based extraction is simpler to read, test, and debug than a nest of CSS selectors.

4.3 Index Constants -- Never Use Magic Numbers

```
parsers/forebet.py -- index constants

# Instead of writing tokens[4] everywhere, we define named constants:

FOOTBALL_LEAGUE_INDEX = 0
FOOTBALL_HOME_TEAM_INDEX = 1
FOOTBALL_AWAY_TEAM_INDEX = 2
FOOTBALL_EVENT_DATETIME_INDEX = 3
FOOTBALL_PROB_1_INDEX = 4 # home win probability %
FOOTBALL_PROB_X_INDEX = 5 # draw probability %
FOOTBALL_PROB_2_INDEX = 6 # away win probability %
FOOTBALL_PRED_OUTCOME_INDEX = 7
FOOTBALL_PREDICTED_HOME_SCORE_INDEX = 8
FOOTBALL_DASH_SEPARATOR_INDEX = 9 # always "-" if row is well-formed
FOOTBALL_PREDICTED_AWAY_SCORE_INDEX = 10
FOOTBALL_CORRECT_SCORE_TEXT_INDEX = 11
FOOTBALL_AVG_GOALS_INDEX = 12
FOOTBALL_WEATHER_INDEX = 13
FOOTBALL_COEF_1_INDEX = 14
FOOTBALL_COEF_X_INDEX = 15
FOOTBALL_COEF_2_INDEX = 16
FOOTBALL_COEF_EXTRA_INDEX = 17
FOOTBALL_MINIMUM_EXPECTED_TOKENS = 18 # reject rows shorter than this
```

WHY

Named constants mean the code reads like documentation. `tokens[FOOTBALL_PROB_1_INDEX]` communicates intent; `tokens[4]` does not. If Forebet ever changes its row structure and inserts a new column, you only update the constant -- not every line of parsing code.

4.4 `parse_forebet_football()` -- The Main Function

```
def parse_forebet_football(html: str) -> list[ForebetFootballPrediction]:
    soup = BeautifulSoup(html, "html.parser")
    rows = soup.select("div.rcnt") # select ALL match rows
    if not rows:
        return [] # empty list -- never None, never raise

    predictions = []
    for row in rows:
```

```

row_text = row.get_text(" | ", strip=True)
parsed = parse_football_row(row_text) # parse one row
if parsed is not None: # skip rows we couldn't parse
    predictions.append(parsed)
return predictions

def parse_football_row(row_text: str) -> ForebetFootballPrediction | None:
    tokens = [t.strip() for t in row_text.split("|") if t.strip()]
    if len(tokens) < FOOTBALL_MINIMUM_EXPECTED_TOKENS:
        return None # row is too short -- probably a header row

    # Confidence check: the dash at index 9 confirms row structure
    confidence = 1.0 if tokens[FOOTBALL_DASH_SEPARATOR_INDEX] == "-" else 0.75

    return ForebetFootballPrediction(
        source="forebet", sport="football",
        league=tokens[FOOTBALL_LEAGUE_INDEX],
        home_team=tokens[FOOTBALL_HOME_TEAM_INDEX],
        # ... all other fields by index ...
        remaining_tokens=tokens[18:], # everything we haven't named yet
        raw_text=row_text,
        confidence=confidence,
    )

```

4.5 Confidence Scoring

Every parsed record carries a confidence score between 0.0 and 1.0. This is not machine learning -- it is a simple structural check.

```

# The dash separator at index 9 acts as a structural anchor.
# In a well-formed upcoming match row:
# tokens[9] == "-" (the score separator between home and away predicted score)
# In a live or finished match, the score may appear there instead.

if tokens[FOOTBALL_DASH_SEPARATOR_INDEX] == "-":
    confidence = 1.0 # structure confirmed -- all field positions are reliable
else:
    confidence = 0.75 # structure may have shifted -- treat with caution

```

**NOT
E**

A confidence of 0.75 does not mean the row is wrong -- it means the row structure did not match our training snapshot exactly. The record is still returned and stored. The signal engine will later use confidence to weight how strongly to trust this record's fields.

5

The SportPesa Parser

src/project_bob/parsers/sportpesa.py -- combining JSON-LD and DOM rows

5.1 The Two-Layer Problem

SportPesa's page is more complex than Forebet's. The page stores data in two separate places that must be combined to get a complete record.

Layer	Contains	Format
JSON-LD scripts	Team names, league, game URL, game ID	Structured JSON inside <code><script type='application/ld+json'></code>
Rendered DOM rows	Date/time, all betting odds values	Text inside <code>.event-row-container</code> divs

WHY

JSON-LD is structured data that search engines use. SportPesa embeds it for SEO -- it is machine-readable and reliable. However it does NOT contain odds. The odds live only in the rendered DOM. So we extract identity from JSON-LD and odds from the DOM, then join them.

5.2 Extracting Fixture Identity from JSON-LD

```
parsers/sportpesa.py -- JSON-LD extraction

def _extract_json_ld_fixtures(soup):
    fixtures = []
    for script in soup.select('script[type="application/ld+json"]'):
        payload = json.loads(script.string or script.get_text())
        # Only process SportsEvent type records
        if isinstance(payload, dict) and payload.get("@type") == "SportsEvent":
            parsed = _parse_json_ld_record(payload)
            if parsed: fixtures.append(parsed)
    return fixtures

def _parse_json_ld_record(payload):
    name = payload.get("name", "") # e.g. "Arsenal - Chelsea"
    if " - " not in name: return None # skip non-fixture records
    home_team, away_team = name.split(" - ", 1)
    url = payload.get("url")

    # Extract game ID from the URL pattern: /games/12345/markets
    game_id_match = re.search(r"/games/(\d+)/markets", str(url or ""))

    # League comes from the location.address.name JSON-LD field
    location = payload.get("location") or {}
```

```
league = (location.get("address") or {}).get("name", "")

return {"home_team": home_team.strip(), "away_team": away_team.strip(),
"league": league, "game_url": url,
"game_url_id": game_id_match.group(1) if game_id_match else None}
```

5.3 Extracting Odds from the DOM with Regex

The rendered odds rows are collected as text strings, then parsed with a compiled regular expression that captures all ten odds values in one pass.

```
parsers/sportpesa.py -- odds regex

# The regex matches the entire rendered row text in one shot.
# Each named group captures one odds value.

pattern = re.compile(
    r"(?P<datetime>\d{2}/\d{2}/\d{2}\s+~\s+\d{2}:\d{2})\s+|\s+"
    r"ID:\s*(?P<game_id>\d+)\s+.*?"
    r"(?P<home_odds>\d+\.\d{2})\s+Draw\s+(?P<draw_odds>\d+\.\d{2})"
    # ... pattern continues for double chance, over/under, BTTS ...
    r"NO\s+\(NG\)\s+(?P<btts_no_odds>\d+\.\d{2})",
    flags=re.IGNORECASE,
)

match = pattern.search(normalized_row_text)
if match:
    return {
        "event_datetime": match.group("datetime"),
        "game_id": match.group("game_id"),
        "home_odds": float(match.group("home_odds")),
        # ... all other odds values ...
    }
```

WHY

A regex that captures everything in one pass is faster and simpler than selecting 10 separate DOM elements. The SportPesa DOM is deeply nested and has inconsistent class names for individual odds cells. The text of the whole row, once normalised, is consistent and reliable.

5.4 Joining the Two Layers

```
def parse_sportpesa_football(html: str) -> list[SportPesaFootballOdds]:
    soup = BeautifulSoup(html, "html.parser")

    # Layer 1: fixture identity from JSON-LD
    fixtures = _extract_json_ld_fixtures(soup)

    # Layer 2: odds from rendered DOM rows
    event_rows = _extract_event_row_texts(soup)
    parsed_rows = [r for row in event_rows
```



```
if (r := _parse_event_row_text(row)) is not None]
# V1 join: by page position (both lists are in the same order)
pair_count = min(len(fixtures), len(parsed_rows))

return [
    SportPesaFootballOdds(
        source="sportpesa", sport="football",
        league=fixtures[i].get("league", "Unknown"),
        home_team=fixtures[i]["home_team"],
        away_team=fixtures[i]["away_team"],
        **parsed_rows[i], # all odds fields from the DOM layer
        game_url=fixtures[i]["game_url"],
        confidence=0.9,
    )
    for i in range(pair_count)
]
```

**WAR
N**

The V1 join is positional -- it assumes fixtures[i] and parsed_rows[i] refer to the same match. This is true on a standard page load but could break if SportPesa ever renders fixtures in a different order than JSON-LD emits them. A future V2 parser should join by game_id instead.

6

The Polymarket API Client

`src/project_bob/scrapers/polymarket.py + parsers/polymarket.py`

6.1 Why API and Not Browser for Polymarket?

Polymarket is fundamentally different from Forebet and SportPesa. It operates a public prediction market that exposes a documented REST API called the Gamma API.

Factor	Forebet/SportPesa	Polymarket
Data access	Browser scraping required	Public REST API (Gamma)
Data format	HTML -- must parse DOM	Structured JSON
Anti-bot	Cloudflare -- headless blocked	None -- API is open
Reliability	Selectors can break if site redesigns	API contract is stable
Speed	15-30s per source (browser launch)	Under 2s (HTTP request)

6.2 The Gamma API Endpoints

```
scrapers/polymarket.py -- API fetch

GAMMA_API_BASE_URL = "https://gamma-api.polymarket.com"

# Two endpoints matter to us:
# GET /markets?limit=100&active=true&closed=false
# Returns market records: question, outcomes, prices, volume, liquidity
# GET /events?limit=500&active=true&closed=false
# Returns event records: category tags, event description, related markets
# We need BOTH because:
# - Markets contain the betting information (prices, outcomes)
# - Events contain the category tags (Sports, Politics, Crypto, etc.)
# - A market's event_id links it to its parent event for enrichment

def _fetch_json(endpoint, params):
    query_string = urlencode(params)
    url = f"{GAMMA_API_BASE_URL}{endpoint}?{query_string}"
    request = Request(url, headers={
        "Accept": "application/json",
        "User-Agent": "project-bob/0.1", # polite identification
    })
    with urlopen(request, timeout=30.0) as response:
```

```
return json.loads(response.read()).decode("utf-8"))
```

6.3 The Normalizer -- parsers/polymarket.py

After fetching raw JSON, `normalize_polymarket_markets()` converts it to `PolymarketMarket` objects. The key challenge is that markets and events are separate payloads that must be joined.

```
def normalize_polymarket_markets(raw_markets, raw_events, category=None,
                                result_limit=None):
    # Step 1: Build an index of events by their ID for fast lookups
    event_map = {str(e["id"]): e for e in raw_events if e.get("id")}

    # Step 2: Normalise each market, enriching it with its parent event data
    normalized = [normalize_market_record(m, event_map) for m in raw_markets]

    # Step 3: Optionally filter by category (e.g. only sports markets)
    if category:
        normalized = [m for m in normalized
                      if matches_requested_category(m, category)]

    # Step 4: Limit results
    return normalized[:result_limit] if result_limit else normalized

def normalize_market_record(raw_record, event_map):
    event_id = extract_event_id(raw_record) # find parent event
    event = event_map.get(event_id or "", {}) # look it up (O(1))
    event_tags = _parse_tags(event.get("tags")) # extract category tags
    derived_category = derive_category(event_tags) # e.g. "Sports"

    return PolymarketMarket(
        source="polymarket",
        market_id=str(raw_record.get("id", "")),
        question=str(raw_record.get("question") or ""),
        category=raw_record.get("category") or derived_category,
        outcomes=_parse_json_list(raw_record.get("outcomes")),
        outcome_prices=[float(p) for p in _parse_json_list(raw_record.get("outcomePrices"))
                        if p is not None],
        raw_record=raw_record, # always keep the original
        confidence=1.0,
        # ... all other fields ...
    )
```

NOTE

`derive_category()` maps event tags to one of the `PRIMARY_CATEGORY_TAGS` (Sports, Politics, Crypto, Finance, etc.). If an event has more than one primary tag it gets labelled 'Mixed' rather than picking arbitrarily. This keeps category data honest and queryable.

7

Inspection Scripts & Smoke Tests

scripts/ -- verifying every scraper works before building on top of it

Why Scripts Matter

The scraper modules are libraries -- they do not run themselves. The scripts/ folder contains short command-line programs that import those libraries and exercise them against live sites. Each script answers one question: does this piece of the pipeline actually work right now?

WHY

Running against a live site is the only way to know whether a site has changed its HTML structure, blocked your IP, or started serving a CAPTCHA. Unit tests with saved HTML snapshots tell you the parser works; smoke tests tell you the scraper still connects.

scripts/smoke_fetch.py -- The Browser Smoke Test

This script instantiates the browser engine and fetches one URL. It prints the BrowserFetchResult fields so you can see the HTTP status, page title, HTML length, any warnings, and whether a snapshot was saved. If anything is broken at the Playwright layer you will see it immediately here before you have to dig through parser logs.

```
scripts/smoke_fetch.py

from ganji_mtaani_agent.scrapers.browser import fetch_page
from ganji_mtaani_agent.scrapers.sources import get_source_config, get_source_target

config = get_source_config("forebet")
target = get_source_target("forebet", "football")

result = fetch_page(
    target.url,
    timeout_ms=config.timeout_ms,
    wait_until=config.wait_until,
    settle_ms=config.settle_ms,
    headless=config.headless,
    snapshot_path="snapshots/forebet_smoke.html",
    screenshot_path="snapshots/forebet_smoke.png",
)

print("ok :", result.ok)
print("status :", result.status)
print("title :", result.title)
print("html_length :", result.html_length)
print("duration_ms :", result.duration_ms)
print("warnings :", result.warnings)
print("error :", result.error)
```

TIP

Always run `smoke_fetch.py` after any environment change -- new machine, new IP address, updated Playwright, or changed proxy settings. It is the fastest way to confirm the browser layer is healthy.

scripts/inspect_forebet_snapshot.py -- Parser Inspection

This script loads a previously saved HTML snapshot from disk and runs the Forebet parser against it without opening a browser. It prints every extracted prediction row so you can verify the tokeniser is reading the right columns. This is the main debugging tool when the parser starts returning empty results or wrong values.

```
scripts/inspect_forebet_snapshot.py

from pathlib import Path
from ganji_mtaani_agent.parsers.forebet import parse_forebet_football

html = Path("snapshots/forebet_smoke.html").read_text(encoding="utf-8")
predictions = parse_forebet_football(html)

print(f"Extracted {len(predictions)} predictions")
for p in predictions[:5]: # show first five rows
    print(" league :", p.league)
    print(" home  :", p.home_team)
    print(" away  :", p.away_team)
    print(" score  :", p.predicted_score)
    print(" avg_goals:", p.avg_goals)
    print(" confidence:", p.confidence)
    print(" remaining:", p.remaining_tokens)
    print()
```

NOTE

`remaining_tokens` is the key debugging field. It contains every token that was NOT assigned to a named field. If a site update adds a new column you will see it appear in `remaining_tokens` before you write a named index for it. This is intentional -- the parser degrades gracefully rather than crashing.

scripts/inspect_sportpesa_snapshot.py -- Two-Layer Debug

SportPesa uses a two-layer extraction approach: JSON-LD for fixture identity and DOM regex for odds. This script loads a SportPesa snapshot and runs both layers independently so you can see whether a failure is in layer one, layer two, or the join step.

```
scripts/inspect_sportpesa_snapshot.py

from pathlib import Path
from ganji_mtaani_agent.parsers.sportpesa import (
    _extract_json_ld_fixtures,
    _extract_event_row_texts,
    parse_sportpesa_football,
)
```

```

html = Path("snapshots/sportpesa_smoke.html").read_text(encoding="utf-8")

fixtures = _extract_json_ld_fixtures(html)
row_texts = _extract_event_row_texts(html)

print(f"JSON-LD fixtures : {len(fixtures)}")
print(f"DOM event rows : {len(row_texts)}")
print()

# Full parse (joined)
results = parse_sportpesa_football(html)
print(f"Joined records : {len(results)}")
for r in results[:3]:
    print(" game_id:", r.game_id)
    print(" home :", r.home_team)
    print(" away :", r.away_team)
    print(" 1X2 :", r.odds_home_win, r.odds_draw, r.odds_away_win)
    print()

```

WAR N

If fixtures and row_texts counts do not match, the positional join will silently truncate to the shorter list. This is logged as a warning but does not crash the pipeline. A mismatch usually means the site structure has changed or a section of matches was hidden behind a login wall.

scripts/fetch_polymarket.py -- API Smoke Test

Polymarket uses the Gamma REST API rather than a browser. This script calls the API directly with a small result_limit so you can verify connectivity, inspect raw market records, and check that the event enrichment join is working correctly.

```

scripts/fetch_polymarket.py

from ganji_mtaani_agent.scrapers.polymarket import fetch_polymarket_markets
from ganji_mtaani_agent.models.polymarket_fetch import PolymarketFetchConfig

config = PolymarketFetchConfig(result_limit=5, active_only=True)
markets = fetch_polymarket_markets(config)

print(f"Fetched {len(markets)} markets")
for m in markets:
    print(" id :", m.market_id)
    print(" question:", m.question[:60])
    print(" category:", m.category)
    print(" outcomes:", m.outcome_map())
    print(" volume :", m.volume)
    print()

```

Reading Snapshot Files

Every BrowserFetchResult can optionally save a snapshot of the raw HTML and a screenshot PNG. These files are written to the snapshots/ directory (which you create locally -- it is not committed to git).

Snapshots are the most important debugging resource you have. When a parser breaks in production, load the snapshot from that run and reproduce the failure locally without needing to hit the live site again.

File	Purpose
snapshots/forebet_smoke.html	Raw HTML from the last Forebet fetch
snapshots/forebet_smoke.png	Screenshot of what Playwright rendered
snapshots/sportpesa_smoke.html	Raw HTML from the last SportPesa fetch
snapshots/sportpesa_smoke.png	Screenshot for visual debugging

TIP

Add snapshots/ to your .gitignore. Snapshots can be megabytes of HTML and change on every run. They are local debugging tools, not source code.

8

What Comes Next

Phase 2: Database | Streamlit UI | Signal Engine | Telegram & WhatsApp

Where We Are Now

The first phase of BOB is complete. You have a working browser engine, a source registry, three scrapers (Forebet, SportPesa, Polymarket), three parsers, and inspection scripts that let you verify each layer independently. Data flows from live sites into clean Python dataclass objects in memory. The next phase connects that in-memory data to a database, adds a visual interface, generates betting signals, and delivers them to users.

Phase 2 Overview

Module	What it does	Key technology
Database layer	Store every scraper run, parsed event, and market snapshot	PostgreSQL + SQLAlchemy
Streamlit UI	Run scrapers, inspect results, review errors -- all from a browser	Streamlit
Signal engine	Convert parsed odds/probabilities into Gold/Silver/Bronze betting signals	Pure Python + Scoring
Decision agent	Explain why a signal exists, flag risks, summarise for users	OpenAI function calling
Telegram delivery	Push signals and summaries to a Telegram channel or group	python-telegram-bot
WhatsApp delivery	Push signals to a WhatsApp number or group	WhatsApp Business API

Step 2A -- Database Layer

The database layer uses SQLAlchemy with a PostgreSQL backend. Every scraper run is recorded in a scraper_runs table so you can see the history of what was fetched, when, and whether it succeeded. Parsed predictions and market records land in events and market_snapshots tables so you can query trends over time.

```
src/ganji_mtaani_agent/db/models.py (planned)

from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
from sqlalchemy import String, Float, DateTime, Text, Integer
import datetime

class Base(DeclarativeBase):
    pass

class ScraperRun(Base):
    __tablename__ = "scraper_runs"
    id : Mapped[int] = mapped_column(Integer, primary_key=True)
    source : Mapped[str] = mapped_column(String(50))
    target : Mapped[str] = mapped_column(String(100))
    started_at : Mapped[datetime.datetime] = mapped_column(DateTime)
```



```

status : Mapped[str] = mapped_column(String(20)) # ok | failed
row_count : Mapped[int] = mapped_column(Integer, default=0)
error : Mapped[str|None] = mapped_column(Text, nullable=True)

class MarketSnapshot(Base):
    __tablename__ = "market_snapshots"
    id : Mapped[int] = mapped_column(Integer, primary_key=True)
    run_id : Mapped[int] = mapped_column(Integer) # FK -> scraper_runs.id
    source : Mapped[str] = mapped_column(String(50))
    market_id : Mapped[str] = mapped_column(String(100))
    question : Mapped[str] = mapped_column(Text)
    category : Mapped[str] = mapped_column(String(100))
    volume : Mapped[float] = mapped_column(Float, default=0.0)
    liquidity : Mapped[float] = mapped_column(Float, default=0.0)
    fetched_at : Mapped[datetime.datetime] = mapped_column(DateTime)

```

WHY

SQLAlchemy's `mapped_column()` with Python type hints (`Mapped[str]`) gives you type-checked models that work with both async and sync sessions. This approach, called the 'ORM 2.0 style', was introduced in SQLAlchemy 2.0 and is the current best practice. It removes boilerplate and makes the schema readable as plain Python.

Step 2B -- Streamlit Scraper UI

The Streamlit UI is an internal operations dashboard. It lets you pick a source and target from dropdowns, click Run Scraper, and immediately see the extracted rows, any warnings, and the raw snapshot. Building this early turns an invisible background process into something you can see, inspect, and demo.

```
src/ganji_mtaani_agent/ui/scraper_ui.py (planned)
```

```

import streamlit as st
from ganji_mtaani_agent.scrapers.sources import SOURCES, get_source_target
from ganji_mtaani_agent.scrapers.browser import fetch_page
from ganji_mtaani_agent.parsers.forebet import parse_forebet_football

st.title("BOB -- Scraper Dashboard")

source_name = st.selectbox("Source", list(SOURCES.keys()))
config = SOURCES[source_name]
target_name = st.selectbox("Target", [t.name for t in config.targets])

if st.button("Run Scraper"):
    target = get_source_target(source_name, target_name)
    with st.spinner("Fetching..."):
        result = fetch_page(target.url,
                             timeout_ms=config.timeout_ms,
                             settle_ms=config.settle_ms,
                             headless=config.headless)
        if result.ok:
            st.success(f"Fetches {result.html_length:,} bytes in {result.duration_ms}ms")

```

```

predictions = parse_forebet_football(result.html)
st.dataframe([vars(p) for p in predictions])
else:
    st.error(f"Scraper failed: {result.error}")

```

Step 2C -- Signal Engine

The signal engine converts parsed records into ranked betting recommendations. The first version is deliberately simple and explainable: convert implied probability from odds, compare across sources, apply a basic edge formula, and assign a tier. No machine learning yet -- the goal is an output a human can audit.

```

src/ganji_mtaani_agent/signals/engine.py (planned)

from dataclasses import dataclass

@dataclass
class BettingSignal:
    event_id : str
    home_team : str
    away_team : str
    selection : str # e.g. "Home Win"
    odds : float
    implied_prob: float # 1 / odds
    edge_score : float # implied_prob - model_prob (if model available)
    tier : str # Gold | Silver | Bronze
    rationale : str # human-readable explanation

    def implied_probability(decimal_odds: float) -> float:
        if decimal_odds <= 0:
            return 0.0
        return round(1.0 / decimal_odds, 4)

    def assign_tier(edge_score: float) -> str:
        if edge_score >= 0.10:
            return "Gold"
        elif edge_score >= 0.05:
            return "Silver"
        else:
            return "Bronze"

```

NOTE

Edge score is the key concept. If the market implies a 40% probability (odds of 2.50) but your model estimates 52%, the edge is +12%. Gold signals are high-edge bets where the market appears to be mispricing the outcome. The tier system makes the output actionable without requiring the user to interpret raw numbers.

Step 2D -- Decision Agent

The Decision Agent wraps the signal engine output with natural-language reasoning. It receives structured signal records and asks the LLM to explain the signal, flag any missing data, identify risk factors, and produce a summary suitable for the dashboard or Telegram channel. The agent never generates signals itself -- it only explains them.

```
src/ganji_mtaani_agent/agents/decision.py (planned)

from openai import OpenAI
from ganji_mtaani_agent.signals.engine import BettingSignal

client = OpenAI()

SYSTEM_PROMPT = """
You are BOB, a Kenyan sports betting analytics assistant.
You receive structured betting signal data and produce a clear,
honest explanation for the user. Never invent data. Never promise wins.
Always flag uncertainty when confidence is low.
"""

def explain_signal(signal: BettingSignal) -> str:
    user_msg = f"""
Signal for: {signal.home_team} vs {signal.away_team}
Selection : {signal.selection}
Odds : {signal.odds}
Edge score: {signal.edge_score:.2%}
Tier : {signal.tier}
Rationale : {signal.rationale}

Write a 3-sentence explanation of this signal for a betting subscriber.
"""
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": user_msg},
        ],
    )
    return response.choices[0].message.content
```

Step 2E -- Telegram & WhatsApp Delivery

Signals and agent summaries are delivered through two channels. Telegram is simpler to set up (create a bot via BotFather, get a token, and use python-telegram-bot). WhatsApp requires a Meta Business Account and uses the Cloud API to send messages to approved templates or open conversations. Both are added after the signal engine is stable.

```
src/ganji_mtaani_agent/delivery/telegram_publisher.py (planned)

import asyncio
from telegram import Bot
```

```
async def send_signal(token: str, chat_id: str, message: str) -> None:
    async with Bot(token) as bot:
        await bot.send_message(chat_id=chat_id, text=message, parse_mode="Markdown")

# Usage:
# asyncio.run(send_signal(
#     token = os.environ["TELEGRAM_BOT_TOKEN"],
#     chat_id = os.environ["TELEGRAM_CHAT_ID"],
#     message = explained_signal_text,
# ))
```

TIP

Create a private Telegram group for testing first. Add your bot as an admin. Send a test message before wiring it to the signal engine. Use /getUpdates on the Telegram API to find your chat_id.

Architecture Decisions to Revisit in Phase 2

Three decisions made in Phase 1 will need to be reviewed as the system grows.

Decision	Phase 1 choice	Phase 2 consideration
Parser join strategy	Positional (index match between JSON-LD and DOM tree)	Does not require me_id match when SportPesa exposes consistent IDs in b
Scraper scheduling	Manual -- run scripts by hand	Add a scheduler (APScheduler or cron) so scrapes run on a timetable
Agent model	Not yet wired	Start with gpt-4o-mini for cost control; upgrade to gpt-4o for production

Safety Principle

BOB is an analytics tool, not a guaranteed profit machine. Every signal the system generates should be accompanied by a confidence level and a clear explanation of what data it is based on. The Decision Agent should never promise wins, never hide uncertainty, and never generate a signal without a traceable source record. If the scraper failed, the signal should not be generated. If the confidence is low, the summary should say so.

WAR
N

Automated betting execution is explicitly out of scope. BOB generates signals and explanations. The human user makes the final decision. This boundary must be maintained even when the system is performing well.

End of Developer Guide

This guide covered the complete Phase 1 codebase of Project BOB -- Bonga na BOB. Every file was explained with its purpose, the reasoning behind the design decision, and a walkthrough of how the key functions work. Phase 2 modules (database, Streamlit UI, signal engine, agents, delivery) will be documented in a companion update to this guide as each module is built and tested.

Build order reminder: scraper -> parser -> database -> UI -> signals -> agent -> delivery. Do not skip steps. The data must be clean before signals can be trusted. Signals must be trusted before the agent explains them. The agent must be stable before delivery is wired.